tools such as AbiWord and Gnumeric, this package is no longer part of Debian, with the individual packages now standing on their own.

Language-specific packs for LibreOffice are distributed in separate packages, most notably *libreoffice-l10n-** and *libreoffice-help-**. Some features such as spelling dictionaries, hyphenation patterns and thesauri are in separate packages, such as *myspell-**, *hunspell-**, *hyphen-** and *mythes-**.

13.9. Emulating Windows: Wine

In spite of all the previously mentioned efforts, there are still a number of tools without a Linux equivalent, or for which the original version is absolutely required. This is where Windows emulation systems come in handy. The most well-known among them is Wine.

➨ `https://www.winehq.org/`

COMPLEMENTS

CrossOver Linux

CrossOver, produced by CodeWeavers, is a set of enhancements to Wine that broadens the available set of emulated features to a point at which Microsoft Office becomes fully usable. Some of the enhancements are periodically merged into Wine.

➨ `https://www.codeweavers.com/products/`

However, one should keep in mind that it is only a solution among others, and the problem can also be tackled with a virtual machine or VNC; both of these solutions are detailed in the sidebars "Virtual machines" page 397 and "*Windows Terminal Server* or VNC" page 397.

Let us start with a reminder: emulation allows executing a program (developed for a target system) on a different host system. The emulation software uses the host system, where the application runs, to imitate the required features of the target system.

Now let's install the required packages (*ttf-mscorefonts-installer* is in the contrib section):

```
# apt install wine ttf-mscorefonts-installer
```

On a 64 bit (amd64) system, if your Windows applications are 32 bit applications, then you will have to enable multi-arch to be able to install wine32 from the i386 architecture (see section 5.4.5, "Multi-Arch Support" page 101).

The user then needs to run `winecfg` and configure which (Debian) locations are mapped to which (Windows) drives. `winecfg` has some sane defaults and can autodetect some more drives; note that even if you have a dual-boot system, you should not point the C: drive at where the Windows partition is mounted in Debian, as Wine is likely to overwrite some of the data on that partition, making Windows unusable. Other settings can be kept to their default values. To run Windows programs, you will first need to install them by running their (Windows) installer under Wine, with a command such as `wine .../setup.exe`; once the program is installed, you can run it with `wine .../program.exe`. The exact location of the `program.exe` file depends

on where the C: drive is mapped; in many cases, however, simply running `wine` *`program`* will work, since the program is usually installed in a location where Wine will look for it by itself.

TIP

Working around a winecfg failure

In some cases, `winecfg` (which is just a wrapper) might fail. As a workaround, it is possible to try to run the underlying command manually: `wine64 /usr/lib/x86_64-linux-gnu/wine/wine/winecfg.exe.so` or `wine32 /usr/lib/i386-linux-gnu/wine/wine/winecfg.exe.so`.

Note that you should not rely on Wine (or similar solutions) without actually testing the particular software: only a real-use test will determine conclusively whether emulation is fully functional.

ALTERNATIVE

Virtual machines

An alternative to emulating Microsoft's operating system is to actually run it in a virtual machine that emulates a full hardware machine. This allows running any operating system. chapter 12, "Advanced Administration" page 328 describes several virtualization systems, most notably Xen and KVM (but also QEMU, VMWare and Bochs).

ALTERNATIVE

***Windows Terminal Server* or VNC**

Yet another possibility is to remotely run the legacy Windows applications on a central server with *Windows Terminal Server* and access the application from Linux machines using *rdesktop*. This is a Linux client for the RDP protocol (*Remote Desktop Protocol*) that *Windows NT/2000 Terminal Server* uses to display desktops on remote machines.

The VNC software provides similar features, with the added benefit of also working with many operating systems. Linux VNC clients and servers are described in section 9.2, "Remote Login" page 207.

13.10. Real-Time Communications software

Debian provides a wide range of Real-Time Communications (RTC) client software. The setup of RTC servers is discussed in section 11.8, "Real-Time Communication Services" page 319. In SIP (Session Initiation Protocol) terminology, a client application or device is also referred to as a user agent.

Each client application varies in functionality. Some applications are more convenient for intensive chat users while other applications are more stable for webcam users. It may be necessary to test several applications to identify those which are most satisfactory. A user may finally decide that they need more than one application, for example, an XMPP application for messaging with customers and an IRC application for collaboration with some online communities.

To maximize the ability of users to communicate with the wider world, it is recommended to configure both SIP and XMPP clients or a single client that supports both protocols.

The default GNOME desktop suggests the Empathy communications client. Empathy can support both SIP and XMPP. It supports instant messaging (IM), voice and video. The KDE project

provides KDE Telepathy, a communications client based on the same underlying Telepathy APIs used by the GNOME Empathy client.

Popular alternatives to Empathy/Telepathy include Ekiga, Linphone, Psi and Jami (formerly known as Ring).

Some of these applications can also interact with mobile users using apps such as Lumicall on Android.

➨ https://lumicall.org

The *Real-Time Communications Quick Start Guide* has a chapter dedicated to client software.

➨ http://rtcquickstart.org/guide/multi/useragents.html

TIP

Look for clients supporting ICE and TURN

Some RTC clients have significant problems sending voice and video through firewalls and NAT networks. Users may receive ghost calls (their phone rings but they don't hear the other person) or they may not be able to call at all.

The ICE and TURN protocols were developed to resolve these issues. Operating a TURN server with public IP addresses in each site and using client software that supports both ICE and TURN gives the best user experience.

If the client software is only intended for instant messaging, there is no requirement for ICE or TURN support.

Debian Developers operate a community SIP service at rtc.debian.org[2]. The community maintains a wiki with documentation about setting up many of the client applications packaged in Debian. The wiki articles and screenshots are a useful resource for anybody setting up a similar service on their own domain.

➨ https://wiki.debian.org/UnifiedCommunications/DebianDevelopers/UserGuide

ALTERNATIVE

Internet Relay Chat

IRC can also be considered, in addition to SIP and XMPP. IRC is more oriented around the concept of channels, the name of which starts with a hash sign #. Each channel is usually targeted at a specific topic and any number of people can join a channel to discuss it (but users can still have one-to-one private conversations if needed). The IRC protocol is older, and does not allow end-to-end encryption of the messages; it is still possible to encrypt the communications between the users and the server by tunneling the IRC protocol inside SSL.

IRC clients are a bit more complex, and they usually provide many features that are of limited use in a corporate environment. For instance, channel "operators" are users endowed with the ability to kick other users from a channel, or even ban them permanently, when the normal discussion is disrupted.

Since the IRC protocol is very old, many clients are available to cater for many user groups; examples include XChat, and Smuxi (graphical clients based on GTK+), Irssi (text mode), Circe (integrated to Emacs), and so on.

[2] https://rtc.debian.org



Keywords

Firewall
Netfilter
nftables
IDS/NIDS





Chapter

14 Security

Contents

Defining a Security Policy 402 Firewall or Packet Filtering 403
Supervision: Prevention, Detection, Deterrence 410 Introduction to AppArmor 417 Introduction to SELinux 424
Other Security-Related Considerations 435 Dealing with a Compromised Machine 440

An information system can have a varying level of importance depending on the environment. In some cases, it is vital to a company's survival. It must therefore be protected from various kinds of risks. The process of evaluating these risks, defining and implementing the protection is collectively known as the "security process".

14.1. Defining a Security Policy

CAUTION

Scope of this chapter

Security is a vast and very sensitive subject, so we cannot claim to describe it in any kind of comprehensive manner in the course of a single chapter. We will only delineate a few important points and describe some of the tools and methods that can be of use in the security domain. For further reading, literature abounds, and entire books have been devoted to the subject. An excellent starting point would be *Linux Server Security* by Michael D. Bauer (published by O'Reilly).

The word "security" itself covers a vast range of concepts, tools and procedures, none of which apply universally. Choosing among them requires a precise idea of what your goals are. Securing a system starts with answering a few questions. Rushing headlong into implementing an arbitrary set of tools runs the risk of focusing on the wrong aspects of security.

The very first thing to determine is therefore the goal. A good approach to help with that determination starts with the following questions:

- *What* are we trying to protect? The security policy will be different depending on whether we want to protect computers or data. In the latter case, we also need to know which data.
- What are we trying to protect *against*? Is it leakage of confidential data? Accidental data loss? Revenue loss caused by disruption of service?
- Also, *who* are we trying to protect against? Security measures will be quite different for guarding against a typo by a regular user of the system than they would be when protecting against a determined attacker group.

The term "risk" is customarily used to refer collectively to these three factors: what to protect, what needs to be prevented from happening, and who will try to make it happen. Modeling the risk requires answers to these three questions. From this risk model, a security policy can be constructed, and the policy can be implemented with concrete actions.

NOTE

Permanent questioning

Bruce Schneier, a world expert in security matters (not only computer security) tries to counter one of security's most important myths with a motto: "Security is a process, not a product". Assets to be protected change in time, and so do threats and the means available to potential attackers. Even if a security policy has initially been perfectly designed and implemented, one should never rest on one's laurels. The risk components evolve, and the response to that risk must evolve accordingly.

Extra constraints are also worth taking into account, as they can restrict the range of available policies. How far are we willing to go to secure a system? This question has a major impact on the policy to implement. The answer is too often only defined in terms of monetary costs, but the other elements should also be considered, such as the amount of inconvenience imposed on system users or performance degradation.

Once the risk has been modeled, one can start thinking about designing an actual security policy.

NOTE

Extreme policies

There are cases where the choice of actions required to secure a system is extremely simple.

For instance, if the system to be protected only comprises a second-hand computer, the sole use of which is to add a few numbers at the end of the day, deciding not to do anything special to protect it would be quite reasonable. The intrinsic value of the system is low. The value of the data is zero since they are not stored on the computer. A potential attacker infiltrating this "system" would only gain an unwieldy calculator. The cost of securing such a system would probably be greater than the cost of a breach.

At the other end of the spectrum, we might want to protect the confidentiality of secret data in the most comprehensive way possible, trumping any other consideration. In this case, an appropriate response would be the total destruction of these data (securely erasing the files, shredding of the hard disks to bits, then dissolving these bits in acid, and so on). If there is an additional requirement that data must be kept in store for future use (although not necessarily readily available), and if cost still isn't a factor, then a starting point would be storing the data on iridium–platinum alloy plates stored in bomb-proof bunkers under various mountains in the world, each of which being (of course) both entirely secret and guarded by entire armies…

Extreme though these examples may seem, they would, nevertheless, be an adequate response to defined risks, insofar as they are the outcome of a thought process that takes into account the goals to reach and the constraints to fulfill. When coming from a reasoned decision, no security policy is less respectable than any other.

In most cases, the information system can be segmented in consistent and mostly independent subsets. Each subsystem will have its own requirements and constraints, and so the risk assessment and the design of the security policy should be undertaken separately for each. A good principle to keep in mind is that a short and well-defined perimeter is easier to defend than a long and winding frontier. The network organization should also be designed accordingly: the sensitive services should be concentrated on a small number of machines, and these machines should only be accessible via a minimal number of check-points; securing these check-points will be easier than securing all the sensitive machines against the entirety of the outside world. It is at this point that the usefulness of network filtering (including by firewalls) becomes apparent. This filtering can be implemented with dedicated hardware, but a possibly simpler and more flexible solution is to use a software firewall such as the one integrated in the Linux kernel.

14.2. Firewall or Packet Filtering

BACK TO BASICS

Firewall

A *firewall* is a piece of computer equipment with hardware and/or software that sorts the incoming or outgoing network packets (coming to or from a local network) and only lets through those matching certain predefined conditions.

A firewall is a filtering network gateway and is only effective on packets that must go through it. Therefore, it can only be effective when going through the firewall is the only route for these packets.

SPECIFIC CASE

Local Firewall

A firewall can be restricted to one particular machine (as opposed to a complete network), in which case its role is to filter or limit access to some services, or possibly to prevent outgoing connections by rogue software that a user could, willingly or not, have installed.

The Linux kernel embeds the *netfilter* firewall, which can be controlled from user space with the `iptables`, `ip6tables`, `arptables` and `ebtables` commands.

However, Netfilter iptables commands are being replaced by nftables, which avoids many of its problems. Its design involves less code duplication, and it can be managed with just the `nft` command. Debian *Buster* uses the nftables framework by default.

To enable a default firewall in Debian execute:

```
# apt install -y nftables
Reading package lists... Done
...
# systemctl enable nftables.service
Created symlink /etc/systemd/system/sysinit.target.wants/nftables.service → /lib/
    ➥ systemd/system/nftables.service.
```

14.2.1. nftables Behavior

As the kernel is processing a network packet it pauses and allows us to inspect the packet and decide what to do with that package. For example, we might want to drop or discard certain incoming packages, modify other packages in various ways, block certain outgoing packets to control against malware or redirect some packets at the earliest possible stage to bridge network interfaces or to spread the load of incoming packets between systems.

A good understanding of the layers 3, 4 and 5 of the OSI (Open Systems Interconnection) model is essential to get the most from netfilter.

CULTURE

The OSI model

The OSI model is a conceptual model to implement networking protocols without regard to its underlying internal structure and technology. Its goal is the interoperability of diverse communication systems with standard communication protocols.

This model was defined in the standard ISO/EIC 7498. The following seven layers are described:

1. Physical: transmission and reception of raw bit streams over a physical medium
2. Data Link: reliable transmission of data frames between two nodes connected by a connected by a physical layer

3. Network: structuring and managing a multi-node network, including addressing, routing and traffic control
4. Transport: reliable transmission of data segments between points on a network, including segmentation, acknowledgment and multiplexing
5. Session: managing communication sessions, i.e. continuous exchange of information in the form of multiple back-and-forth transmissions between two nodes
6. Presentation: translation of data between a networking service and an application; including character encoding, data compression and encryption/decryption
7. Application: High-level APIs, including resource sharing, remote file access.

More information can be found on Wikipedia:

➨ https://en.wikipedia.org/wiki/Osi_model

The firewall is configured with *tables*, which hold *rules* contained in *chains*. Unlike iptables, *nftables* does not have any default table. The user decides which and how many tables to create. Every table must have only one of the following five families assigned: ip, ip6, inet, arp and bridge. ip is used if the family is not specified.

There are two types of chains: *base chains* and *regular chains*. A base chain is an entry point for packets from the networking stack, they are registered into the Netfilter hooks, ie. these chains see packets flowing through the TCP/IP stack. On the other hand, and a regular chain is not attached to any hook, so they do not see any traffic, but it may be used as a jump target for better organization.

Rules are made of statements, which includes some expressions to be matched and then a verdict statement, like accept, drop, queue, continue, return, jump chain and goto chain.

BACK TO BASICS

ICMP

ICMP (*Internet Control Message* Protocol) is the protocol used to transmit complementary information on communications. It allows testing network connectivity with the `ping` command (which sends an ICMP *echo request* message, which the recipient is meant to answer with an ICMP *echo reply* message). It signals a firewall rejecting a packet, indicates an overflow in a receive buffer, proposes a better route for the next packets in the connection, and so on. This protocol is defined by several RFC documents; the initial RFC777 and RFC792 were soon completed and extended.

➨ http://www.faqs.org/rfcs/rfc777.html

➨ http://www.faqs.org/rfcs/rfc792.html

For reference, a receive buffer is a small memory zone storing data between the time it arrives from the network and the time the kernel handles it. If this zone is full, new data cannot be received, and ICMP signals the problem, so that the emitter can slow down its transfer rate (which should ideally reach an equilibrium after some time).

Note that although an IPv4 network can work without ICMP, ICMPv6 is strictly required for an IPv6 network, since it combines several functions that were, in the IPv4 world, spread across ICMPv4, IGMP (*Internet Group Membership Protocol*) and ARP (*Address Resolution Protocol*). ICMPv6 is defined in RFC4443.

➨ http://www.faqs.org/rfcs/rfc4443.html

14.2.2. Moving from iptables to nftables

The `iptables-translate` and `ip6tables-translate` commands can be used to translate old iptables commands into the new nftables syntax. Whole rulesets can also be translated, in this case we migrate the rules configured in one computer which has Docker installed:

```
# iptables-save > iptables-ruleset.txt
# iptables-restore-translate -f iptables-ruleset.txt

# Translated by iptables-restore-translate v1.8.2 on Thu Jul 18 10:39:33 2019
add table ip filter
add chain ip filter INPUT { type filter hook input priority 0; policy accept; }
add chain ip filter FORWARD { type filter hook forward priority 0; policy drop; }
add chain ip filter OUTPUT { type filter hook output priority 0; policy accept; }
add chain ip filter DOCKER
add chain ip filter DOCKER-ISOLATION-STAGE-1
add chain ip filter DOCKER-ISOLATION-STAGE-2
add chain ip filter DOCKER-USER
add rule ip filter FORWARD counter jump DOCKER-USER
add rule ip filter FORWARD counter jump DOCKER-ISOLATION-STAGE-1
add rule ip filter FORWARD oifname "docker0" ct state related,established counter
    ➥ accept
add rule ip filter FORWARD oifname "docker0" counter jump DOCKER
add rule ip filter FORWARD iifname "docker0" oifname != "docker0" counter accept
add rule ip filter FORWARD iifname "docker0" oifname "docker0" counter accept
add rule ip filter DOCKER-ISOLATION-STAGE-1 iifname "docker0" oifname != "docker0"
    ➥ counter jump DOCKER-ISOLATION-STAGE-2
add rule ip filter DOCKER-ISOLATION-STAGE-1 counter return
add rule ip filter DOCKER-ISOLATION-STAGE-2 oifname "docker0" counter drop
add rule ip filter DOCKER-ISOLATION-STAGE-2 counter return
add rule ip filter DOCKER-USER counter return
add table ip nat
add chain ip nat PREROUTING { type nat hook prerouting priority -100; policy accept;
    ➥ }
add chain ip nat INPUT { type nat hook input priority 100; policy accept; }
add chain ip nat POSTROUTING { type nat hook postrouting priority 100; policy accept;
    ➥  }
add chain ip nat OUTPUT { type nat hook output priority -100; policy accept; }
add chain ip nat DOCKER
add rule ip nat PREROUTING fib daddr type local counter jump DOCKER
add rule ip nat POSTROUTING oifname != "docker0" ip saddr 172.17.0.0/16 counter
    ➥ masquerade
add rule ip nat OUTPUT ip daddr != 127.0.0.0/8 fib daddr type local counter jump
    ➥ DOCKER
add rule ip nat DOCKER iifname "docker0" counter return
# Completed on Thu Jul 18 10:39:33 2019
# iptables-restore-translate -f iptables-ruleset.txt > ruleset.nft
# nft -f ruleset.nft
# nft list ruleset
```

```
table ip filter {
        chain INPUT {
                type filter hook input priority 0; policy accept;
        }

        chain FORWARD {
                type filter hook forward priority 0; policy drop;
                counter packets 0 bytes 0 jump DOCKER-USER
                counter packets 0 bytes 0 jump DOCKER-ISOLATION-STAGE-1
                oifname ”docker0” ct state related,established counter packets 0
                    ➥ bytes 0 accept
                oifname ”docker0” counter packets 0 bytes 0 jump DOCKER
                iifname ”docker0” oifname != ”docker0” counter packets 0 bytes 0
                    ➥ accept
                iifname ”docker0” oifname ”docker0” counter packets 0 bytes 0 accept
        }

        chain OUTPUT {
                type filter hook output priority 0; policy accept;
        }

        chain DOCKER {
        }

        chain DOCKER-ISOLATION-STAGE-1 {
                iifname ”docker0” oifname != ”docker0” counter packets 0 bytes 0 jump
                    ➥  DOCKER-ISOLATION-STAGE-2
                counter packets 0 bytes 0 return
        }

        chain DOCKER-ISOLATION-STAGE-2 {
                oifname ”docker0” counter packets 0 bytes 0 drop
                counter packets 0 bytes 0 return
        }

        chain DOCKER-USER {
                counter packets 0 bytes 0 return
        }
}
table ip nat {
        chain PREROUTING {
                type nat hook prerouting priority -100; policy accept;
                fib daddr type local counter packets 0 bytes 0 jump DOCKER
        }

        chain INPUT {
                type nat hook input priority 100; policy accept;
        }
```

```
        chain POSTROUTING {
                type nat hook postrouting priority 100; policy accept;
                oifname != "docker0" ip saddr 172.17.0.0/16 counter packets 0 bytes 0
                    ➥ masquerade
        }

        chain OUTPUT {
                type nat hook output priority -100; policy accept;
                ip daddr != 127.0.0.0/8 fib daddr type local counter packets 0 bytes
                    ➥ 0 jump DOCKER
        }

        chain DOCKER {
                iifname "docker0" counter packets 0 bytes 0 return
        }
}
table ip mangle {
        chain PREROUTING {
                type filter hook prerouting priority -150; policy accept;
        }

        chain INPUT {
                type filter hook input priority -150; policy accept;
        }

        chain FORWARD {
                type filter hook forward priority -150; policy accept;
        }

        chain OUTPUT {
                type route hook output priority -150; policy accept;
        }

        chain POSTROUTING {
                type filter hook postrouting priority -150; policy accept;
        }
}
```

The tools `iptables-nft`, `ip6tables-nft`, `arptables-nft`, `ebtables-nft` are versions of iptables that use the nftables API, so users can keep using the old iptables syntax with them, but that is not recommended; these tools should only be used for backwards compatibility.

14.2.3. Syntax of `nft`

The `nft` commands allow manipulating tables, chains and rules. The table option supports multiple operations: add, create, delete, list and flush. `nft add table ip6 mangle` adds a new table from the family ip6.

To insert a new base chain to the filter table, you can execute the following command (note that the semicolon is escaped with a backslash when using Bash):

```
# nft add chain filter input { type filter hook input priority 0 \; }
```

Rules are usually added with the following syntax: `nft add rule [`*`family`*`] `*`table chain`* `handle `*`handle`* `statement`.

`insert` is similar to the `add` command, but the given rule is prepended to the beginning of the chain or before the rule with the given handle instead of at the end or after that rule. For example, the following command inserts a rule before the rule with handler number 8:

```
# nft insert rule filter output position 8 ip daddr 127.0.0.8 drop
```

The executed `nft` commands do not make permanent changes to the configuration, so they are lost if they are not saved. The firewall rules are located in `/etc/nftables.conf`. A simple way to save the current firewall configuration permanently is to execute `nft list ruleset > /etc/nftables.conf` as root.

`nft` allows many more operations, refer to its manual page `nft(8)` for more information.

14.2.4. Installing the Rules at Each Boot

To enable a default firewall in Debian, you need to store the rules in `/etc/nftables.conf` and execute `systemctl enable nftables.service` as root. You can stop the firewall executing `nft flush ruleset` as root.

In other cases, the recommended way is to register the configuration script in `up` directive of the `/etc/network/interfaces` file. In the following example, the script is stored under `/usr/local/etc/arrakis.fw`.

Example 14.1 *`interfaces` file calling firewall script*

```
auto eth0
iface eth0 inet static
    address 192.168.0.1
    network 192.168.0.0
    netmask 255.255.255.0
    broadcast 192.168.0.255
    up /usr/local/etc/arrakis.fw
```

This obviously assumes that you are using *ifupdown* to configure the network interfaces. If you are using something else (like *NetworkManager* or *systemd-networkd*), then refer to their respective documentation to find out ways to execute a script after the interface has been brought up.

14.3. Supervision: Prevention, Detection, Deterrence

Monitoring is an integral part of any security policy for several reasons. Among them, that the goal of security is usually not restricted to guaranteeing data confidentiality, but it also includes ensuring availability of the services. It is therefore imperative to check that everything works as expected, and to detect in a timely manner any deviant behavior or change in quality of the service(s) rendered. Monitoring activity can help detecting intrusion attempts and enable a swift reaction before they cause grave consequences. This section reviews some tools that can be used to monitor several aspects of a Debian system. As such, it completes section 12.4, "Monitoring" page 371.

14.3.1. Monitoring Logs with `logcheck`

The `logcheck` program monitors log files every hour by default. It sends unusual log messages in emails to the administrator for further analysis.

The list of monitored files is stored in `/etc/logcheck/logcheck.logfiles`; the default values work fine if the `/etc/rsyslog.conf` file has not been completely overhauled.

`logcheck` can work in one of three more or less detailed modes: *paranoid*, *server* and *workstation*. The first one is *very* verbose, and should probably be restricted to specific servers such as firewalls. The second (and default) mode is recommended for most servers. The last one is designed for workstations, and is even terser (it filters out more messages).

In all three cases, `logcheck` should probably be customized to exclude some extra messages (depending on installed services), unless the admin really wishes to receive hourly batches of long uninteresting emails. Since the message selection mechanism is rather complex, `/usr/share/doc/logcheck-database/README.logcheck-database.gz` is a required — if challenging — read.

The applied rules can be split into several types:

- those that qualify a message as a cracking attempt (stored in a file in the `/etc/logcheck/cracking.d/` directory);
- those canceling such a qualification (`/etc/logcheck/cracking.ignore.d/`);
- those classifying a message as a security alert (`/etc/logcheck/violations.d/`);
- those canceling this classification (`/etc/logcheck/violations.ignore.d/`);
- finally, those applying to the remaining messages (considered as *system events*).

CAUTION

Ignoring a message

Any message tagged as a cracking attempt or a security alert (following a rule stored in a `/etc/logcheck/violations.d/myfile` file) can only be ignored by a rule in a `/etc/logcheck/violations.ignore.d/myfile` or `/etc/logcheck/violations.ignore.d/myfile-`*`extension`* file.

A system event is always signaled unless a rule in one of the `/etc/logcheck/ignore.d.{paranoid,server,workstation}/` directories states the event should be ignored. Of course, the only directories taken into account are those corresponding to verbosity levels equal or greater than the selected operation mode.

14.3.2. Monitoring Activity

In Real Time

`top` is an interactive tool that displays a list of currently running processes. The default sorting is based on the current amount of processor use and can be obtained with the P key. Other sort orders include a sort by occupied memory (M key), by total processor time (T key) and by process identifier (N key). The k key allows killing a process by entering its process identifier. The r key allows *renicing* a process, i.e. changing its priority.

When the system seems to be overloaded, `top` is a great tool to see which processes are competing for processor time or consume too much memory. In particular, it is often interesting to check if the processes consuming resources match the real services that the machine is known to host. An unknown process running as the www-data user should really stand out and be investigated, since it is probably an instance of software installed and executed on the system through a vulnerability in a web application.

`top` is a very flexible tool and its manual page gives details on how to customize its display and adapt it to one's personal needs and habits.

The `gnome-system-monitor` graphical tool is similar to `top` and it provides roughly the same features.

History

Processor load, network traffic and free disk space are information that are constantly varying. Keeping a history of their evolution is often useful in determining exactly how the computer is used.

There are many dedicated tools for this task. Most can fetch data via SNMP (*Simple Network Management Protocol*) in order to centralize this information. An added benefit is that this allows fetching data from network elements that may not be general-purpose computers, such as dedicated network routers or switches.

This book deals with Munin in some detail (see section 12.4.1, "Setting Up Munin" page 372) as part of Chapter 12: "Advanced Administration" page 328. Debian also provides a similar tool, *cacti*. Its deployment is slightly more complex, since it is based solely on SNMP. Despite having a web interface, grasping the concepts involved in configuration still requires some effort. Reading the HTML documentation (`/usr/share/doc/cacti/html/Table-of-Contents.html`) should be considered a prerequisite.

ALTERNATIVE

mrtg

`mrtg` (in the similarly-named package) is an older tool. Despite some rough edges, it can aggregate historical data and display them as graphs. It includes a number of scripts dedicated to collecting the most commonly monitored data such as processor load, network traffic, web page hits, and so on.

The *mrtg-contrib* and *mrtgutils* packages contain example scripts that can be used directly.

14.3.3. Avoiding Intrusion

Attackers try to get access to servers by guessing passwords, which is why strong passwords must always be used. Even then, you should also establish measures against brute-force attacks. A brute-force attack is an attempt to log in to an unauthorised software system by performing multiple login attempts in a short period of time.

The best way to stop a brute-force atack is to limit the number of login attempts coming from the same origin, usually by temporarily banning an IP address.

Fail2Ban is an intrusion prevention software suite that can be configured to monitor any service that writes login attemps to a log file. It can be found in the package *fail2ban*.

Fail2Ban is configured through a simple protocol by `fail2ban-client`, which also reads configuration files and issues corresponding configuration commands to the server, `fail2ban-server`. It has four configuration file types, all stored in `/etc/fail2ban`:

- `fail2ban.conf`. Global configuration (such as logging).
- `filter.d/*.conf`. Filters specifying how to detect authentication failures. The Debian package already contains filters for many common programs.
- `action.d/*.conf`. Actions defining the commands for banning and unbanning of IP addresses.
- `jail.conf`. It is where *jails*, the combinations of filters and actions, are defined.

Let us have a look at the configuration of `sshd` in `/etc/fail2ban/jail.conf` to better understand how Fail2Ban works...

```
[...]
[DEFAULT]
[...]
bantime  = 10m
[...]
maxretry = 5
[...]
[sshd]
port    = ssh
logpath = %(sshd_log)s
backend = %(sshd_backend)s
```

Fail2Ban will check for failed login attepts for `sshd` using Python regular expressions defined in `/etc/fail2ban/filters.d/sshd.conf` against the log file of `sshd`, which is defined in the variable sshd_log in the file `/etc/fail2ban/paths_common.conf`. If Fail2Ban detects five failed login attempts in a row, it will ban the IP address where those attempts originated.

Fail2Ban is a very simple and effective way to protect against the most common brute-force attacks, but it cannot protect against distributed brute-force attacks, which is when an attacker uses a large number of machines spread around the Internet.

A good way to provide extra protection against distributed brute force attacks is to artificially increase the login time after each failed attempt.

14.3.4. Detecting Changes

Once the system is installed and configured, and barring security upgrades, there is usually no reason for most of the files and directories to evolve, data excepted. It is therefore interesting to make sure that files actually do not change: any unexpected change would therefore be worth investigating. This section presents a few tools able to monitor files and to warn the administrator when an unexpected change occurs (or simply to list such changes).

Auditing Packages with `dpkg --verify`

GOING FURTHER

Protecting against upstream changes

`dpkg --verify` is useful in detecting changes to files coming from a Debian package, but it will be useless if the package itself is compromised, for instance, if the Debian mirror is compromised. Protecting against this class of attacks involves using APT's digital signature verification system (see section 6.6, "Checking Package Authenticity" page 132), and taking care to only install packages from a certified origin.

`dpkg --verify` (or `dpkg -V`) is an interesting tool since it allows finding what installed files have been modified (potentially by an attacker), but this should be taken with a grain of salt. To do its job it relies on checksums stored in dpkg's own database which is stored on the hard disk (they can be found in `/var/lib/dpkg/info/`*`package`*`.md5sums`); a thorough attacker will therefore update these files so they contain the new checksums for the subverted files.

BACK TO BASICS

File fingerprint

As a reminder: a fingerprint is a value, often a number (even though in hexadecimal notation), that contains a kind of signature for the contents of a file. This signature is calculated with an algorithm (MD5 or SHA1 being well-known examples) that more or less guarantee that even the tiniest change in the file contents implies a change in the fingerprint; this is known as the "avalanche effect". This allows a simple numerical fingerprint to serve as a litmus test to check whether the contents of a file have been altered. These algorithms are not reversible; in other words, for most of them, knowing a fingerprint doesn't allow finding the corresponding contents. Recent mathematical advances seem to weaken the absoluteness of these principles, but their use is not called into question so far, since creating different contents yielding the same fingerprint still seems to be quite a difficult task.

Running `dpkg -V` will verify all installed packages and will print out a line for each file with a failing test. The output format is the same as the one of `rpm -V` where each character denotes a test on some specific meta-data. Unfortunately `dpkg` does not store the meta-data needed for most tests and will thus output question marks for them. Currently only the checksum test can yield a "5" on the third character (when it fails).

```
# dpkg -V
??5?????? /lib/systemd/system/ssh.service
??5?????? c /etc/libvirt/qemu/networks/default.xml
??5?????? c /etc/lvm/lvm.conf
??5?????? c /etc/salt/roster
```

In the sample above, dpkg reports a change to SSH's service file that the administrator made to the packaged file instead of using an appropriate `/etc/systemd/system/ssh.service` override (which would be stored below `/etc` like any configuration change should be). It also lists multiple configuration files (identified by the "c" letter on the second field) that had been legitimately modified.

Auditing Packages: debsums and its Limits

`debsums` is the ancestor of `dpkg -V` and is thus mostly obsolete. It suffers from the same limitations than dpkg. Fortunately, some of the limitations can be worked-around (whereas dpkg does not offer similar work-arounds).

Since the data on the disk cannot be trusted, `debsums` offers to do its checks based on `.deb` files instead of relying on dpkg's database. To download trusted `.deb` files of all the packages installed, we can rely on APT's authenticated downloads. This operation can be slow and tedious, and should therefore not be considered a proactive technique to be used on a regular basis.

```
# apt-get --reinstall -d install 'grep-status -e 'Status: install ok installed' -n -s
    ➥ Package'
[ ... ]
# debsums -p /var/cache/apt/archives --generate=all
```

Note that this example uses the `grep-status` command from the *dctrl-tools* package, which is not installed by default.

`debsums` can be run frequently as a cronjob setting CRON_CHECK in `/etc/default/debsums`. To ignore certain files outside the `/etc` directory, which have been altered on purpuse or which are expected to change (like `/usr/share/misc/pci.ids`) you can add them to `/etc/debsums-ignore`.

Monitoring Files: AIDE

The AIDE tool (*Advanced Intrusion Detection Environment*) allows checking file integrity, and detecting any change against a previously recorded image of the valid system. This image is

stored as a database (`/var/lib/aide/aide.db`) containing the relevant information on all files of the system (fingerprints, permissions, timestamps and so on). This database is first initialized with `aideinit`; it is then used daily (by the `/etc/cron.daily/aide` script) to check that nothing relevant changed. When changes are detected, AIDE records them in log files (`/var/log/aide/*.log`) and sends its findings to the administrator by email.

IN PRACTICE

Protecting the database

Since AIDE uses a local database to compare the states of the files, the validity of its results is directly linked to the validity of the database. If an attacker gets root permissions on a compromised system, they will be able to replace the database and cover their tracks. A possible workaround would be to store the reference data on read-only storage media.

Many options in `/etc/default/aide` can be used to tweak the behavior of the *aide* package. The AIDE configuration proper is stored in `/etc/aide/aide.conf` and `/etc/aide/aide.conf.d/` (actually, these files are only used by `update-aide.conf` to generate `/var/lib/aide/aide.conf.autogenerated`). Configuration indicates which properties of which files need to be checked. For instance, the contents of log files changes routinely, and such changes can be ignored as long as the permissions of these files stay the same, but both contents and permissions of executable programs must be constant. Although not very complex, the configuration syntax is not fully intuitive, and reading the `aide.conf(5)` manual page is therefore recommended.

A new version of the database is generated daily in `/var/lib/aide/aide.db.new`; if all recorded changes were legitimate, it can be used to replace the reference database.

ALTERNATIVE

Tripwire and Samhain

Tripwire is very similar to AIDE; even the configuration file syntax is almost the same. The main addition provided by *tripwire* is a mechanism to sign the configuration file, so that an attacker cannot make it point at a different version of the reference database.

Samhain also offers similar features, as well as some functions to help detecting rootkits (see the sidebar "The *checksecurity* and *chkrootkit*/*rkhunter* packages" page 415). It can also be deployed globally on a network, and record its traces on a central server (with a signature).

QUICK LOOK

The *checksecurity* and *chkrootkit*/*rkhunter* packages

The first of these packages contains several small scripts performing basic checks on the system (empty passwords, new setuid files, and so on) and warning the administrator if required. Despite its explicit name, an administrator should not rely solely on it to make sure a Linux system is secure.

The *chkrootkit* and *rkhunter* packages allow looking for *rootkits* potentially installed on the system. As a reminder, these are pieces of software designed to hide the compromise of a system while discreetly keeping control of the machine. The tests are not 100% reliable, but they can usually draw the administrator's attention to potential problems.

rkhunter also performs checks to see if commands have been modified, if the system startup files have been modified, and various checks on the network interfaces, including checks for listening applications.

BACK TO BASICS

Denial of service

A "denial of service" attack has only one goal: to make a service unavailable. Whether such an attack involves overloading the server with queries or exploiting a bug, the end result is the same: the service is no longer operational. Regular users are unhappy, and the entity hosting the targeted network service suffers a loss in reputation (and possibly in revenue, for instance if the service was an e-commerce site).

Such an attack is sometimes "distributed"; this usually involves overloading the server with large numbers of queries coming from many different sources so that the server becomes unable to answer the legitimate queries. These types of attacks have gained well-known acronyms: DDoS and DoS (depending on whether the denial of service attack is distributed or not).

`suricata` (in the Debian package of the same name) is a NIDS — a *Network Intrusion Detection System*. Its function is to listen to the network and try to detect infiltration attempts and/or hostile acts (including denial of service attacks). All these events are logged in multiple files in `/var/log/suricata`. There are third party tools (Kibana/logstash) to better browse all the data collected.

➨ `https://suricata-ids.org`

➨ `https://www.elastic.co/products/kibana`

CAUTION

Range of action

The effectiveness of `suricata` is limited by the traffic seen on the monitored network interface. It will obviously not be able to detect anything if it cannot observe the real traffic. When plugged into a network switch, it will therefore only monitor attacks targeting the machine it runs on, which is probably not the intention. The machine hosting `suricata` should therefore be plugged into the "mirror" port of the switch, which is usually dedicated to chaining switches and therefore gets all the traffic.

Configuring suricata involves reviewing and editing `/etc/suricata/suricata-debian.yaml`, which is very long because each parameter is abundantly commented. A minimal configuration requires describing the range of addresses that the local network covers (HOME_NET parameter). In practice, this means the set of all potential attack targets. But getting the most of it requires reading it in full and adapting it to the local situation.

On top of this, you should also edit `/etc/default/suricata` to define the network interface to monitor and to enable the init script (by setting RUN=yes). You might also want to set LISTENMODE=pcap because the default LISTENMODE=nfqueue requires further configuration to work properly (the netfilter firewall must be configured to pass packets to some user-space queue handled by suricata via the NFQUEUE target).

To detect bad behavior, `suricata` needs a set of monitoring rules: you can find such rules in the *snort-rules-default* package. `snort` is the historical reference in the IDS ecosystem and `suricata` is able to reuse rules written for it.

Alternatively, `oinkmaster` (in the package of the same name) can be used to download Snort rulesets from external sources.

GOING FURTHER

Integration with `prelude`

Prelude brings centralized monitoring of security information. Its modular architecture includes a server (the *manager* in *prelude-manager*) which gathers alerts generated by *sensors* of various types.

Suricata can be configured as such a sensor. Other possibilities include *prelude-lml* (*Log Monitor Lackey*) which monitors log files (in a manner similar to `logcheck`, described in section 14.3.1, "Monitoring Logs with `logcheck`" page 410).

14.4. Introduction to AppArmor

14.4.1. Principles

AppArmor is a *Mandatory Access Control* (MAC) system built on Linux's LSM (*Linux Security Modules*) interface. In practice, the kernel queries AppArmor before each system call to know whether the process is authorized to do the given operation. Through this mechanism, AppArmor confines programs to a limited set of resources.

AppArmor applies a set of rules (known as "profile") on each program. The profile applied by the kernel depends on the installation path of the program being executed. Contrary to SELinux (discussed in section 14.5, "Introduction to SELinux" page 424), the rules applied do not depend on the user. All users face the same set of rules when they are executing the same program (but traditional user permissions still apply and might result in different behavior!).

AppArmor profiles are stored in `/etc/apparmor.d/` and they contain a list of access control rules on resources that each program can make use of. The profiles are compiled and loaded into the kernel by the `apparmor_parser` command. Each profile can be loaded either in enforcing or complaining mode. The former enforces the policy and reports violation attempts, while the latter does not enforce the policy but still logs the system calls that would have been denied.

14.4.2. Enabling AppArmor and managing AppArmor profiles

AppArmor support is built into the standard kernels provided by Debian. Enabling AppArmor is thus just a matter of installing some packages by executing `apt install apparmor apparmor-profiles apparmor-utils` with root privileges.

AppArmor is functional after the installation, and `aa-status` will confirm it quickly:

```
# aa-status
apparmor module is loaded.
40 profiles are loaded.
23 profiles are in enforce mode.
   /usr/bin/evince
   /usr/bin/evince-previewer
```

```
[...]
17 profiles are in complain mode.
   /usr/sbin/dnsmasq
[...]
14 processes have profiles defined.
12 processes are in enforce mode.
   /usr/bin/evince (3462)
[...]
2 processes are in complain mode.
   /usr/sbin/avahi-daemon (429) avahi-daemon
   /usr/sbin/avahi-daemon (511) avahi-daemon
0 processes are unconfined but have a profile defined.
```

NOTE **More AppArmor profiles**

The *apparmor-profiles* package contains profiles managed by the upstream AppArmor community. To get even more profiles you can install *apparmor-profiles-extra* which contains profiles developed by Ubuntu and Debian.

The state of each profile can be switched between enforcing and complaining with calls to `aa-enforce` and `aa-complain` giving as parameter either the path of the executable or the path to the policy file. Additionally a profile can be entirely disabled with `aa-disable` or put in audit mode (to log accepted system calls too) with `aa-audit`.

```
# aa-enforce /usr/bin/pidgin
Setting /usr/bin/pidgin to enforce mode.
# aa-complain /usr/sbin/dnsmasq
Setting /usr/sbin/dnsmasq to complain mode.
```

14.4.3. Creating a new profile

Even though creating an AppArmor profile is rather easy, most programs do not have one. This section will show you how to create a new profile from scratch just by using the target program and letting AppArmor monitor the system call it makes and the resources it accesses.

The most important programs that need to be confined are the network facing programs as those are the most likely targets of remote attackers. That is why AppArmor conveniently provides an `aa-unconfined` command to list the programs which have no associated profile and which expose an open network socket. With the --paranoid option you get all unconfined processes that have at least one active network connection.

```
# aa-unconfined
801 /sbin/dhclient not confined
409 /usr/sbin/NetworkManager not confined
411 /usr/sbin/cupsd confined by '/usr/sbin/cupsd (enforce)'
429 /usr/sbin/avahi-daemon confined by 'avahi-daemon (enforce)'
516 /usr/sbin/cups-browsed confined by '/usr/sbin/cups-browsed (enforce)'
538 /usr/sbin/zebra not confined
```

```
591 /usr/sbin/named not confined
847 /usr/sbin/mysqld not confined
849 /usr/sbin/sshd not confined
1013 /usr/sbin/dhclient (/sbin/dhclient) not confined
1276 /usr/sbin/apache2 not confined
1322 /usr/sbin/apache2 not confined
1323 /usr/sbin/apache2 not confined
1324 /usr/sbin/apache2 not confined
1325 /usr/sbin/apache2 not confined
1327 /usr/sbin/apache2 not confined
1829 /usr/lib/ipsec/charon confined by '/usr/lib/ipsec/charon (enforce)'
2132 /usr/sbin/exim4 not confined
12865 /usr/bin/python3.7 (/usr/bin/python3) not confined
12873 /usr/bin/python3.7 (/usr/bin/python3) not confined
```

In the following example, we will thus try to create a profile for `/sbin/dhclient`. For this we will use `aa-genprof dhclient`. In Debian *Buster* there is a known bug[1] that makes the previous command fail with the following error: `ERROR: Include file /etc/apparmor.d/local/usr.lib.dovecot.deliver not found`. To fix it create the missing files with `touch` *`file`*. It will invite you to use the application in another window and when done to come back to `aa-genprof` to scan for AppArmor events in the system logs and convert those logs into access rules. For each logged event, it will make one or more rule suggestions that you can either approve or further edit in multiple ways:

```
# aa-genprof dhclient
Writing updated profile for /usr/sbin/dhclient.
Setting /usr/sbin/dhclient to complain mode.

Before you begin, you may wish to check if a
profile already exists for the application you
wish to confine. See the following wiki page for
more information:
https://gitlab.com/apparmor/apparmor/wikis/Profiles

Profiling: /usr/sbin/dhclient

Please start the application to be profiled in
another window and exercise its functionality now.

Once completed, select the "Scan" option below in
order to scan the system logs for AppArmor events.

For each AppArmor event, you will be given the
opportunity to choose whether the access should be
allowed or denied.

[(S)can system log for AppArmor events] / (F)inish
```

[1] https://bugs.debian.org/cgi-bin/bugreport.cgi?bug=928160

```
Reading log entries from /var/log/syslog.
Updating AppArmor profiles in /etc/apparmor.d.
```

```
Profile:  /usr/sbin/dhclient ❶
Execute:  /usr/sbin/dhclient-script
Severity: unknown

(I)nherit / (C)hild / (P)rofile / (N)amed / (U)nconfined / (X) ix On / (D)eny / Abo(r
    ➥ )t / (F)inish
P
Should AppArmor sanitise the environment when
switching profiles?

Sanitising environment is more secure,
but some applications depend on the presence
of LD_PRELOAD or LD_LIBRARY_PATH.

(Y)es / [(N)o]
Y
Writing updated profile for /usr/sbin/dhclient-script.
Complain-mode changes:
```

```
Profile:    /usr/sbin/dhclient ❷
Capability: net_raw
Severity:   8

 [1 - capability net_raw,]
[(A)llow] / (D)eny / (I)gnore / Audi(t) / Abo(r)t / (F)inish
A
Adding capability net_raw to profile.

Profile:  /sbin/dhclient
Capability: net_bind_service
Severity:   8

 [1 - #include <abstractions/nis> ]
  2 - capability net_bind_service,
(A)llow / [(D)eny] / (I)gnore / Audi(t) / Abo(r)t / (F)inish
A
Adding #include <abstractions/nis> to profile.
```

```
Profile:  /usr/sbin/dhclient ❸
Path:     /etc/ssl/openssl.cnf
New Mode: owner r
Severity: 2

 [1 - #include <abstractions/lightdm>]
  2 - #include <abstractions/openssl>
  3 - #include <abstractions/ssl_keys>
```

```
  4 - owner /etc/ssl/openssl.cnf r,
(A)llow / [(D)eny] / (I)gnore / (G)lob / Glob with (E)xtension / (N)ew / Audi(t) / (O
    ➥ )wner permissions off / Abo(r)t / (F)inish
2

Profile:  /usr/sbin/dhclient
Path:     /etc/ssl/openssl.cnf
New Mode: owner r
Severity: 2

  1 - #include <abstractions/lightdm>
 [2 - #include <abstractions/openssl>]
  3 - #include <abstractions/ssl_keys>
  4 - owner /etc/ssl/openssl.cnf r,
[(A)llow] / (D)eny / (I)gnore / (G)lob / Glob with (E)xtension / (N)ew / Abo(r)t / (F
    ➥ )inish / (M)ore
A
[...]
```

```
Profile:  /usr/sbin/dhclient-script ❹
Path:     /usr/bin/dash
New Mode: owner r
Severity: unknown

 [1 - #include <abstractions/lightdm>]
  2 - #include <abstractions/ubuntu-browsers.d/plugins-common>
  3 - owner /usr/bin/dash r,
(A)llow / [(D)eny] / (I)gnore / (G)lob / Glob with (E)xtension / (N)ew / Audi(t) / (O
    ➥ )wner permissions off / Abo(r)t / (F)inish
A
Adding #include <abstractions/lightdm> to profile.
Deleted 2 previous matching profile entries.

= Changed Local Profiles =

The following local profiles were changed. Would you like to save them?

 [1 - /usr/sbin/dhclient]
  2 - /usr/sbin/dhclient-script
(S)ave Changes / Save Selec(t)ed Profile / [(V)iew Changes] / View Changes b/w (C)
    ➥ lean profiles / Abo(r)t
S
Writing updated profile for /usr/sbin/dhclient.
Writing updated profile for /usr/sbin/dhclient-script.

Profiling: /usr/sbin/dhclient

Please start the application to be profiled in
another window and exercise its functionality now.
```

```
Once completed, select the "Scan" option below in
order to scan the system logs for AppArmor events.

For each AppArmor event, you will be given the
opportunity to choose whether the access should be
allowed or denied.

[(S)can system log for AppArmor events] / (F)inish
F
Reloaded AppArmor profiles in enforce mode.

Please consider contributing your new profile!
See the following wiki page for more information:
https://gitlab.com/apparmor/apparmor/wikis/Profiles

Finished generating profile for /usr/sbin/dhclient.
```

Note that the program does not display back the control characters that you type but for the clarity of the explanation I have included them in the previous transcript.

1. The first event detected is the execution of another program. In that case, you have multiple choices: you can run the program with the profile of the parent process (the "Inherit" choice), you can run it with its own dedicated profile (the "Profile" and the "Named" choices, differing only by the possibility to use an arbitrary profile name), you can run it with a sub-profile of the parent process (the "Child" choice), you can run it without any profile (the "Unconfined" choice) or you can decide to not run it at all (the "Deny" choice).

   Note that when you opt to run it under a dedicated profile that doesn't exist yet, the tool will create the missing profile for you and will make rule suggestions for that profile in the same run.

2. At the kernel level, the special powers of the root user have been split in "capabilities". When a system call requires a specific capability, AppArmor will verify whether the profile allows the program to make use of this capability.

3. Here the program seeks read permissions for `/etc/ssl/openssl.cnf`. `aa-genprof` detected that this permission was also granted by multiple "abstractions" and offers them as alternative choices. An abstraction provides a reusable set of access rules grouping together multiple resources that are commonly used together. In this specific case, the file is generally accessed through the nameservice related functions of the C library and we type "2" to first select the "#include <abstractions/openssl>" choice and then "A" to allow it.

4. Notice that this access request is not part of the dhclient profile but of the new profile that we created when we allowed `/usr/sbin/dhclient-script` to run with its own profile.

After having gone through all the logged events, the program offers to save all the profiles that were created during the run. In this case, we have two profiles that we save at once with "Save" (but you can save them individually too) before leaving the program with "Finish".

`aa-genprof` is in fact only a smart wrapper around `aa-logprof`: it creates an empty profile, loads it in complain mode and then run `aa-logprof` which is a tool to update a profile based on the profile violations that have been logged. So you can re-run that tool later to improve the profile that you just created.

If you want the generated profile to be complete, you should use the program in all the ways that it is legitimately used. In the case of dhclient, it means running it via Network Manager, running it via ifupdown, running it manually, etc. In the end, you might get a `/etc/apparmor.d/usr.sbin.dhclient` close to this:

```
# Last Modified: Fri Jul  5 00:51:02 2019
#include <tunables/global>

/usr/sbin/dhclient {
  #include <abstractions/base>
  #include <abstractions/nameservice>

  capability net_bind_service,
  capability net_raw,

  /bin/dash r,
  /etc/dhcp/* r,
  /etc/dhcp/dhclient-enter-hooks.d/* r,
  /etc/dhcp/dhclient-exit-hooks.d/* r,
  /etc/resolv.conf.* w,
  /etc/samba/dhcp.conf.* w,
  /proc/*/net/dev r,
  /proc/filesystems r,
  /run/dhclient*.pid w,
  /sbin/dhclient mr,
  /sbin/dhclient-script rCx,
  /usr/lib/NetworkManager/nm-dhcp-helper Px,
  /var/lib/NetworkManager/* r,
  /var/lib/NetworkManager/*.lease rw,
  /var/lib/dhcp/*.leases rw,

  owner /etc/** mrwk,
  owner /var/** mrwk,
  owner /{,var/}run/** mrwk,
}
```

And `/etc/apparmor.d/usr.sbin.dhclient-script` might be similar to this:

```
# Last Modified: Fri Jul  5 00:51:55 2019
```

```
#include <tunables/global>

/usr/sbin/dhclient-script {
  #include <abstractions/base>
  #include <abstractions/bash>
  #include <abstractions/lightdm>
}
```

14.5. Introduction to SELinux

14.5.1. Principles

SELinux (*Security Enhanced Linux*) is a *Mandatory Access Control* system built on Linux's LSM (*Linux Security Modules*) interface. In practice, the kernel queries SELinux before each system call to know whether the process is authorized to do the given operation.

SELinux uses a set of rules — collectively known as a *policy* — to authorize or forbid operations. Those rules are difficult to create. Fortunately, two standard policies (*targeted* and *strict*) are provided to avoid the bulk of the configuration work.

With SELinux, the management of rights is completely different from traditional Unix systems. The rights of a process depend on its *security context*. The context is defined by the *identity* of the user who started the process, the *role* and the *domain* that the user carried at that time. The rights really depend on the domain, but the transitions between domains are controlled by the roles. Finally, the possible transitions between roles depend on the identity.

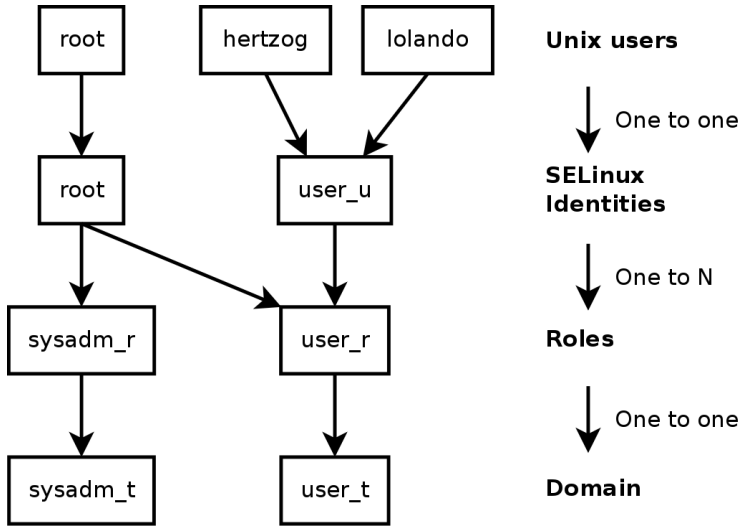


Figure 14.1 *Security contexts and Unix users*

In practice, during login, the user gets assigned a default security context (depending on the roles that they should be able to endorse). This defines the current domain, and thus the domain that all new child processes will carry. If you want to change the current role and its associated domain, you must call `newrole -r role_r -t domain_t` (there is usually only a single domain allowed for a given role, the -t parameter can thus often be left out). This command authenticates you by asking you to type your password. This feature forbids programs to automatically switch roles. Such changes can only happen if they are explicitly allowed in the SELinux policy.

Obviously the rights do not apply to all *objects* (files, directories, sockets, devices, etc.). They can vary from object to object. To achieve this, each object is associated to a *type* (this is known as labeling). Domains' rights are thus expressed with sets of (dis)allowed operations on those types (and, indirectly, on all objects which are labeled with the given type).

EXTRA

Domains and types are equivalent

Internally, a domain is just a type, but a type that only applies to processes. That is why domains are suffixed with _t just like objects' types.

By default, a program inherits its domain from the user who started it, but the standard SELinux policies expect many important programs to run in dedicated domains. To achieve this, those executables are labeled with a dedicated type (for example `ssh` is labeled with ssh_exec_t, and when the program starts, it automatically switches to the ssh_t domain). This automatic domain transition mechanism makes it possible to grant only the rights required by each program. It is a fundamental principle of SELinux.

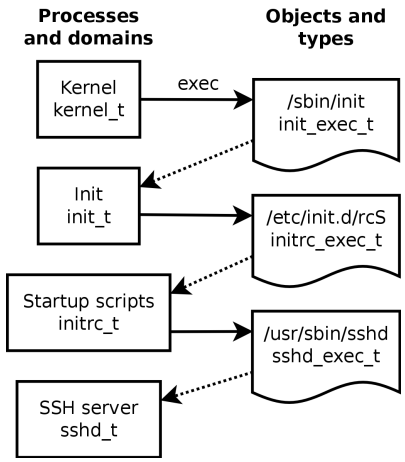


Figure 14.2 *Automatic transitions between domains*

IN PRACTICE

Finding the security context

To find the security context of a given process, you should use the Z option of `ps`.

```
$ ps axZ | grep vstfpd
system_u:system_r:ftpd_t:s0   2094 ?    Ss  0:00 /usr/sbin/
    ➥ vsftpd
```

```
$ id -Z
unconfined_u unconfined_r unconfined_t s0 s0 c0 c1023
```

```
$ ls -Z test /usr/bin/ssh
unconfined_u object_r user_home_t s0 test
     system_u object_r ssh_exec_t s0 /usr/bin/ssh
```

14.5.2. Setting Up SELinux

SELinux support is built into the standard kernels provided by Debian. The core Unix tools support SELinux without any modifications. It is thus relatively easy to enable SELinux.

The `apt install selinux-basics selinux-policy-default` command will automatically install the packages required to configure an SELinux system.

The *selinux-policy-default* package contains a set of standard rules. By default, this policy only restricts access for a few widely exposed services. The user sessions are not restricted and it is thus unlikely that SELinux would block legitimate user operations. However, this does enhance the security of system services running on the machine. To setup a policy equivalent to the old "strict" rules, you just have to disable the unconfined module (modules management is detailed further in this section).

Once the policy has been installed, you should label all the available files (which means assigning them a type). This operation must be manually started with `fixfiles relabel`.

The SELinux system is now ready. To enable it, you should add the selinux=1 security=selinux parameter to the Linux kernel. The audit=1 parameter enables SELinux logging which records all the denied operations. Finally, the enforcing=1 parameter brings the rules into application: without it SELinux works in its default *permissive* mode where denied actions are logged but still executed. You should thus modify the GRUB bootloader configuration file to append the desired parameters. One easy way to do this is to modify the GRUB_CMDLINE_LINUX variable in `/etc/default/grub` and to run `update-grub`. SELinux will be active after a reboot.

It is worth noting that the `selinux-activate` script automates those operations and forces a labeling on next boot (which avoids new non-labeled files created while SELinux was not yet active and while the labeling was going on).

14.5.3. Managing an SELinux System

The SELinux policy is a modular set of rules, and its installation detects and enables automatically all the relevant modules based on the already installed services. The system is thus immediately operational. However, when a service is installed after the SELinux policy, you must be able to manually enable the corresponding module. That is the purpose of the `semodule` command. Furthermore, you must be able to define the roles that each user can endorse, and this can be done with the `semanage` command.

Those two commands can thus be used to modify the current SELinux configuration, which is stored in `/etc/selinux/default/`. Unlike other configuration files that you can find in `/etc/`, all those files must not be changed by hand. You should use the programs designed for this purpose.

GOING FURTHER

More documentation

Since the NSA doesn't provide any official documentation, the community set up a wiki to compensate. It brings together a lot of information, but you must be aware that most SELinux contributors are Fedora users (where SELinux is enabled by default). The documentation thus tends to deal specifically with that distribution.

➨ https://selinuxproject.org

You should also have a look at the dedicated Debian wiki page as well as Russell Coker's blog, who is one of the most active Debian developers working on SELinux support.

➨ https://wiki.debian.org/SELinux

➨ https://etbe.coker.com.au/tag/selinux/

Managing SELinux Modules

Available SELinux modules are stored in the `/usr/share/selinux/default/` directory. To enable one of these modules in the current configuration, you should use `semodule -i` *`module.pp.bz2`*. The *pp.bz2* extension stands for *policy package* (compressed with bzip2).

Removing a module from the current configuration is done with `semodule -r` *`module`*. Finally, the `semodule -l` command lists the modules which are currently installed. It also outputs their version numbers. Modules can be selectively enabled with `semodule -e` and disabled with `semodule -d`.

```
# semodule -i /usr/share/selinux/default/abrt.pp.bz2
libsemanage.semanage_direct_install_info: abrt module will be disabled after install
    ➥ as there is a disabled instance of this module present in the system.
# semodule -l
accountsd
```

```
acct
[...]
# semodule -e abrt
# semodule -d accountsd
# semodule -l
abrt
acct
[...]
# semodule -r abrt
libsemanage.semanage_direct_remove_key: abrt module at priority 100 is now active.
    ➥ semodule -l
```

`semodule` immediately loads the new configuration unless you use its `-n` option. It is worth noting that the program acts by default on the current configuration (which is indicated by the SELINUXTYPE variable in `/etc/selinux/config`), but that you can modify another one by specifying it with the -s option.

Managing Identities

Every time that a user logs in, they get assigned an SELinux identity. This identity defines the roles that they will be able to endorse. Those two mappings (from the user to the identity and from this identity to roles) are configurable with the `semanage` command.

You should definitely read the `semanage(8)` manual page. All the managed concepts have their own manual page; for instance, `semanage-login(8)`. Even if the command's syntax tends to be similar for all the concepts which are managed, it is recommended to read its manual page. You will find common options to most sub-commands: `-a` to add, `-d` to delete, `-m` to modify, `-l` to list, and `-t` to indicate a type (or domain).

`semanage login -l` lists the current mapping between user identifiers and SELinux identities. Users that have no explicit entry get the identity indicated in the `__default__` entry. The `semanage login -a -s user_u` *`user`* command will associate the *user_u* identity to the given user. Finally, `semanage login -d` *`user`* drops the mapping entry assigned to this user.

```
# semanage login -a -s user_u rhertzog
# semanage login -l

Login Name           SELinux User         MLS/MCS Range        Service

__default__          unconfined_u         s0-s0:c0.c1023       *
rhertzog             user_u               s0                   *
root                 unconfined_u         s0-s0:c0.c1023       *
# semanage login -d rhertzog
```

`semanage user -l` lists the mapping between SELinux user identities and allowed roles. Adding a new identity requires to define both the corresponding roles and a labeling prefix which is used to assign a type to personal files (`/home/`*`user`*`/*`). The prefix must be picked among

user, staff, and sysadm. The "staff" prefix results in files of type "staff_home_dir_t". Creating a new SELinux user identity is done with `semanage user -a -R` *`roles`* `-P` *`prefix identity`*. Finally, you can remove an SELinux user identity with `semanage user -d` *`identity`*.

```
# semanage user -a -R 'staff_r user_r' -P staff test_u
# semanage user -l

                Labeling   MLS/       MLS/
SELinux User    Prefix     MCS Level  MCS Range                      SELinux Roles

root            sysadm     s0         s0-s0:c0.c1023                 staff_r sysadm_r
    ➥  system_r
staff_u         staff      s0         s0-s0:c0.c1023                 staff_r sysadm_r
sysadm_u        sysadm     s0         s0-s0:c0.c1023                 sysadm_r
system_u        user       s0         s0-s0:c0.c1023                 system_r
test_u          staff      s0         s0                             staff_r user_r
unconfined_u    unconfined s0         s0-s0:c0.c1023                 system_r
    ➥ unconfined_r
user_u          user       s0         s0                             user_r
# semanage user -d test_u
```

Managing File Contexts, Ports and Booleans

Each SELinux module provides a set of file labeling rules, but it is also possible to add custom labeling rules to cater to a specific case. For example, if you want the web server to be able to read files within the `/srv/www/` file hierarchy, you could execute `semanage fcontext -a -t httpd_sys_content_t "/srv/www(/.*)?"` followed by `restorecon -R /srv/www/`. The former command registers the new labeling rules and the latter resets the file types according to the current labeling rules.

Similarly, TCP/UDP ports are labeled in a way that ensures that only the corresponding daemons can listen to them. For instance, if you want the web server to be able to listen on port 8080, you should run `semanage port -m -t http_port_t -p tcp 8080`.

Some SELinux modules export boolean options that you can tweak to alter the behavior of the default rules. The `getsebool` utility can be used to inspect those options (`getsebool` *`boolean`* displays one option, and `getsebool -a` them all). The `setsebool` *`boolean value`* command changes the current value of a boolean option. The -P option makes the change permanent, it means that the new value becomes the default and will be kept across reboots. The example below grants web servers an access to home directories (this is useful when users have personal websites in `~/public_html/`).

```
# getsebool httpd_enable_homedirs
httpd_enable_homedirs --> off
# setsebool -P httpd_enable_homedirs on
# getsebool httpd_enable_homedirs
httpd_enable_homedirs --> on
```

14.5.4. Adapting the Rules

Since the SELinux policy is modular, it might be interesting to develop new modules for (possibly custom) applications that lack them. These new modules will then complete the *reference policy*.

To create new modules, the *selinux-policy-dev* package is required, as well as *selinux-policy-doc*. The latter contains the documentation of the standard rules (`/usr/share/doc/selinux-policy-doc/html/`) and sample files that can be used as templates to create new modules. Install those files and study them more closely:

```
$ cp /usr/share/doc/selinux-policy-doc/Makefile.example Makefile
$ cp /usr/share/doc/selinux-policy-doc/example.fc ./
$ cp /usr/share/doc/selinux-policy-doc/example.if ./
$ cp /usr/share/doc/selinux-policy-doc/example.te ./
```

The `.te` file is the most important one. It defines the rules. The `.fc` file defines the "file contexts", that is the types assigned to files related to this module. The data within the `.fc` file are used during the file labeling step. Finally, the `.if` file defines the interface of the module: it is a set of "public functions" that other modules can use to properly interact with the module that you're creating.

Writing a `.fc` file

Reading the below example should be sufficient to understand the structure of such a file. You can use regular expressions to assign the same security context to multiple files, or even an entire directory tree.

Example 14.2 *`example.fc` file*

```
# myapp executable will have:
# label: system_u:object_r:myapp_exec_t
# MLS sensitivity: s0
# MCS categories: <none>

/usr/sbin/myapp         --      gen_context(system_u:object_r:myapp_exec_t,s0)
```

Writing a `.if` File

In the sample below, the first interface ("myapp_domtrans") controls who can execute the application. The second one ("myapp_read_log") grants read rights on the application's log files.

Each interface must generate a valid set of rules which can be embedded in a `.te` file. You should thus declare all the types that you use (with the gen_require macro), and use standard directives to grant rights. Note, however, that you can use interfaces provided by other modules. The next section will give more explanations about how to express those rights.

Example 14.3 *example.if File*

```
## <summary>Myapp example policy</summary>
## <desc>
##      <p>
##              More descriptive text about myapp.  The <desc>
##              tag can also use <p>, <ul>, and <ol>
##              html tags for formatting.
##      </p>
##      <p>
##              This policy supports the following myapp features:
##              <ul>
##              <li>Feature A</li>
##              <li>Feature B</li>
##              <li>Feature C</li>
##              </ul>
##      </p>
## </desc>
#

########################################
## <summary>
##      Execute a domain transition to run myapp.
## </summary>
## <param name="domain">
##      Domain allowed to transition.
## </param>
#
interface(`myapp_domtrans',`
        gen_require(`
                type myapp_t, myapp_exec_t;
        ')

        domtrans_pattern($1,myapp_exec_t,myapp_t)
')

########################################
## <summary>
##      Read myapp log files.
## </summary>
## <param name="domain">
##      Domain allowed to read the log files.
## </param>
#
interface(`myapp_read_log',`
        gen_require(`
                type myapp_log_t;
        ')
```

```
        logging_search_logs($1)
        allow $1 myapp_log_t:file r_file_perms;
')
```

DOCUMENTATION

Explanations about the *reference policy*

The *reference policy* evolves like any free software project: based on volunteer contributions. The project is hosted by Tresys, one of the most active companies in the SELinux field. Their wiki contains explanations on how the rules are structured and how you can create new ones.

➨ https://github.com/SELinuxProject/refpolicy/wiki/GettingStarted

Writing a `.te` File

Have a look at the `example.te` file:

GOING FURTHER

The `m4` macro language

To properly structure the policy, the SELinux developers used a macro-command processor. Instead of duplicating many similar *allow* directives, they created "macro functions" to use a higher-level logic, which also results in a much more readable policy.

In practice, `m4` is used to compile those rules. It does the opposite operation: it expands all those high-level directives into a huge database of *allow* directives.

The SELinux "interfaces" are only macro functions which will be substituted by a set of rules at compilation time. Likewise, some rights are in fact sets of rights which are replaced by their values at compilation time.

```
policy_module(myapp,1.0.0) ❶

########################################
#
# Declarations
#

type myapp_t; ❷
type myapp_exec_t;
domain_type(myapp_t)
domain_entry_file(myapp_t, myapp_exec_t) ❸

type myapp_log_t;
logging_log_file(myapp_log_t) ❹

type myapp_tmp_t;
files_tmp_file(myapp_tmp_t)

########################################
```

```
#
# Myapp local policy
#

allow myapp_t myapp_log_t:file { read_file_perms append_file_perms }; ❺

allow myapp_t myapp_tmp_t:file manage_file_perms;
files_tmp_filetrans(myapp_t,myapp_tmp_t,file)
```

❶ The module must be identified by its name and version number. This directive is required.

❷ If the module introduces new types, it must declare them with directives like this one. Do not hesitate to create as many types as required rather than granting too many useless rights.

❸ Those interfaces define the myapp_t type as a process domain that should be used by any executable labeled with myapp_exec_t. Implicitly, this adds an exec_type attribute on those objects, which in turn allows other modules to grant rights to execute those programs: for instance, the userdomain module allows processes with domains user_t, staff_t, and sysadm_t to execute them. The domains of other confined applications will not have the rights to execute them, unless the rules grant them similar rights (this is the case, for example, of `dpkg` with its dpkg_t domain).

❹ logging_log_file is an interface provided by the reference policy. It indicates that files labeled with the given type are log files which ought to benefit from the associated rules (for example, granting rights to `logrotate` so that it can manipulate them).

❺ The allow directive is the base directive used to authorize an operation. The first parameter is the process domain which is allowed to execute the operation. The second one defines the object that a process of the former domain can manipulate. This parameter is of the form "*type*:*class*" where *type* is its SELinux type and *class* describes the nature of the object (file, directory, socket, fifo, etc.). Finally, the last parameter describes the permissions (the allowed operations).

Permissions are defined as the set of allowed operations and follow this template: { *operation1 operation2* }. However, you can also use macros representing the most useful permissions. The `/usr/share/selinux/devel/include/support/obj_perm_sets.spt` lists them.

The following web page provides a relatively exhaustive list of object classes, and permissions that can be granted.

➨ https://selinuxproject.org/page/ObjectClassesPerms

Now you just have to find the minimal set of rules required to ensure that the target application or service works properly. To achieve this, you should have a good knowledge of how the application works and of what kind of data it manages and/or generates.

However, an empirical approach is possible. Once the relevant objects are correctly labeled, you can use the application in permissive mode: the operations that would be forbidden are logged but still succeed. By analyzing the logs, you can now identify the operations to allow. Here is an example of such a log entry:

```
avc:  denied  { read write } for  pid=1876 comm="syslogd" name="xconsole" dev=tmpfs
    ➥ ino=5510 scontext=system_u:system_r:syslogd_t:s0 tcontext=system_u:object_r:
    ➥ device_t:s0 tclass=fifo_file permissive=1
```

To better understand this message, let us study it piece by piece.

Message	Description
`avc: denied`	An operation has been denied.
`{ read write }`	This operation required the read and write permissions.
`pid=1876`	The process with PID 1876 executed the operation (or tried to execute it).
`comm="syslogd"`	The process was an instance of the syslogd program.
`name="xconsole"`	The target object was named xconsole. Sometimes you can also have a "path" variable — with the full path — instead.
`dev=tmpfs`	The device hosting the target object is a tmpfs (an in-memory filesystem). For a real disk, you could see the partition hosting the object (for example, "sda3").
`ino=5510`	The object is identified by the inode number 5510.
`scontext=system_u:system_r:syslogd_t:s0`	This is the security context of the process who executed the operation.
`tcontext=system_u:object_r:device_t:s0`	This is the security context of the target object.
`tclass=fifo_file`	The target object is a FIFO file.

Table 14.1 *Analysis of an SELinux trace*

By observing this log entry, it is possible to build a rule that would allow this operation. For example, allow syslogd_t device_t:fifo_file { read write }. This process can be automated, and it is exactly what the `audit2allow` command (of the *policycoreutils* package) offers. This approach is only useful if the various objects are already correctly labeled according to what must be confined. In any case, you will have to carefully review the generated rules and validate them according to your knowledge of the application. Effectively, this approach tends to grant more rights than are really required. The proper solution is often to create new types and to grant rights on those types only. It also happens that a denied operation isn't fatal to the application,

in which case it might be better to just add a "dontaudit" rule to avoid the log entry despite the effective denial.

COMPLEMENTS

No roles in policy rules

It might seem weird that roles do not appear at all when creating new rules. SELinux uses only the domains to find out which operations are allowed. The role intervenes only indirectly by allowing the user to switch to another domain. SELinux is based on a theory known as *Type Enforcement* and the type is the only element that matters when granting rights.

Compiling the Files

Once the 3 files (`example.if`, `example.fc`, and `example.te`) match your expectations for the new rules, rename them to `myapp.`*`extension`* and run `make NAME=devel` to generate a module in the `myapp.pp` file (you can immediately load it with `semodule -i myapp.pp`). If several modules are defined, `make` will create all the corresponding `.pp` files.

14.6. Other Security-Related Considerations

Security is not just a technical problem; more than anything, it is about good practices and understanding the risks. This section reviews some of the more common risks, as well as a few best practices which should, depending on the case, increase security or lessen the impact of a successful attack.

14.6.1. Inherent Risks of Web Applications

The universal character of web applications led to their proliferation. Several are often run in parallel: a webmail, a wiki, some groupware system, forums, a photo gallery, a blog, and so on. Many of those applications rely on the "LAMP" (*Linux, Apache, MySQL, PHP*) stack. Unfortunately, many of those applications were also written without much consideration for security problems. Data coming from outside is, too often, used with little or no validation. Providing specially-crafted values can be used to subvert a call to a command so that another one is executed instead. Many of the most obvious problems have been fixed as time has passed, but new security problems pop up regularly.

VOCABULARY

SQL injection

When a program inserts data into SQL queries in an insecure manner, it becomes vulnerable to SQL injections; this name covers the act of changing a parameter in such a way that the actual query executed by the program is different from the intended one, either to damage the database or to access data that should normally not be accessible.

➨ https://en.wikipedia.org/wiki/SQL_Injection

Updating web applications regularly is therefore a must, lest any cracker (whether a professional attacker or a script kiddy) can exploit a known vulnerability. The actual risk depends on the case, and ranges from data destruction to arbitrary code execution, including web site defacement.

14.6.2. Knowing What To Expect

A vulnerability in a web application is often used as a starting point for cracking attempts. What follows is a short review of possible consequences.

QUICK LOOK

Filtering HTTP queries

Apache 2 includes modules allowing filtering incoming HTTP queries. This allows blocking some attack vectors. For instance, limiting the length of parameters can prevent buffer overflows. More generally, one can validate parameters before they are even passed to the web application and restrict access along many criteria. This can even be combined with dynamic firewall updates, so that a client infringing one of the rules is banned from accessing the web server for a given period of time.

Setting up these checks can be a long and cumbersome task, but it can pay off when the web application to be deployed has a dubious track record where security is concerned.

mod-security2 (in the *libapache2-mod-security2* package) is the main such module. It even comes with many ready-to-use rules of its own (in the *modsecurity-crs* package) that you can easily enable.

The consequences of an intrusion will have various levels of obviousness depending on the motivations of the attacker. *Script-kiddies* only apply recipes they find on web sites; most often, they deface a web page or delete data. In more subtle cases, they add invisible contents to web pages so as to improve referrals to their own sites in search engines.

A more advanced attacker will go beyond that. A disaster scenario could go on in the following fashion: the attacker gains the ability to execute commands as the www-data user, but executing a command requires many manipulations. To make their life easier, they install other web applications specially designed to remotely execute many kinds of commands, such as browsing the filesystem, examining permissions, uploading or downloading files, executing commands, and even provide a network shell. Often, the vulnerability will allow running a `wget` command that will download some malware into `/tmp/`, then executing it. The malware is often downloaded from a foreign website that was previously compromised, in order to cover tracks and make it harder to find out the actual origin of the attack.

At this point, the attacker has enough freedom of movement that they often install an IRC *bot* (a robot that connects to an IRC server and can be controlled by this channel). This bot is often used to share illegal files (unauthorized copies of movies or software, and so on). A determined attacker may want to go even further. The www-data account does not allow full access to the machine, and the attacker will try to obtain administrator privileges. Now, this should not be possible, but if the web application was not up-to-date, chances are that the kernel and other programs are outdated too; this sometimes follows a decision from the administrator who, de-

spite knowing about the vulnerability, neglected to upgrade the system since there are no local users. The attacker can then take advantage of this second vulnerability to get root access.

VOCABULARY

Privilege escalation

This term covers anything that can be used to obtain more permissions than a given user should normally have. The `sudo` program is designed for precisely the purpose of giving administrative rights to some users. But the same term is also used to describe the act of an attacker exploiting a vulnerability to obtain undue rights.

Now the attacker owns the machine; they will usually try to keep this privileged access for as long as possible. This involves installing a *rootkit*, a program that will replace some components of the system so that the attacker will be able to obtain the administrator privileges again at a later time; the rootkit also tries hiding its own existence as well as any traces of the intrusion. A subverted `ps` program will omit to list some processes, `netstat` will not list some of the active connections, and so on. Using the root permissions, the attacker was able to observe the whole system, but didn't find important data; so they will try accessing other machines in the corporate network. Analyzing the administrator's account and the history files, the attacker finds what machines are routinely accessed. By replacing `sudo` or `ssh` with a subverted program, the attacker can intercept some of the administrator's passwords, which they will use on the detected servers... and the intrusion can propagate from then on.

This is a nightmare scenario which can be prevented by several measures. The next few sections describe some of these measures.

14.6.3. Choosing the Software Wisely

Once the potential security problems are known, they must be taken into account at each step of the process of deploying a service, especially when choosing the software to install. Many web sites, such as SecurityFocus.com, keep a list of recently-discovered vulnerabilities, which can give an idea of a security track record before some particular software is deployed. Of course, this information must be balanced against the popularity of said software: a more widely-used program is a more tempting target, and it will be more closely scrutinized as a consequence. On the other hand, a niche program may be full of security holes that never get publicized due to a lack of interest in a security audit.

VOCABULARY

Security audit

A security audit is the process of thoroughly reading and analyzing the source code of some software, looking for potential security vulnerabilities it could contain. Such audits are usually proactive and they are conducted to ensure a program meets certain security requirements.

In the free software world, there is generally ample room for choice, and choosing one piece of software over another should be a decision based on the criteria that apply locally. More features imply an increased risk of a vulnerability hiding in the code; picking the most advanced

program for a task may actually be counter-productive, and a better approach is usually to pick the simplest program that meets the requirements.

VOCABULARY
Zero-day exploit
A *zero-day exploit* attack is hard to prevent; the term covers a vulnerability that is not yet known to the authors of the program.

14.6.4. Managing a Machine as a Whole

Most Linux distributions install by default a number of Unix services and many tools. In many cases, these services and tools are not required for the actual purposes for which the administrator set up the machine. As a general guideline in security matters, unneeded software is best uninstalled. Indeed, there is no point in securing an FTP server, if a vulnerability in a different, unused service can be used to get administrator privileges on the whole machine.

By the same reasoning, firewalls will often be configured to only allow access to services that are meant to be publicly accessible.

Current computers are powerful enough to allow hosting several services on the same physical machine. From an economic viewpoint, such a possibility is interesting: only one computer to administrate, lower energy consumption, and so on. From the security point of view, however, such a choice can be a problem. One compromised service can bring access to the whole machine, which in turn compromises the other services hosted on the same computer. This risk can be mitigated by isolating the services. This can be attained either with virtualization (each service being hosted in a dedicated virtual machine or container), or with AppArmor/SELinux (each service daemon having an adequately designed set of permissions).

14.6.5. Users Are Players

Discussing security immediately brings to mind protection against attacks by anonymous crackers hiding in the Internet jungle; but an often-forgotten fact is that risks also come from inside: an employee about to leave the company could download sensitive files on the important projects and sell them to competitors, a negligent salesman could leave their desk without locking their session during a meeting with a new prospect, a clumsy user could delete the wrong directory by mistake, and so on.

The response to these risks can involve technical solutions: no more than the required permissions should be granted to users, and regular backups are a must. But in many cases, the appropriate protection is going to involve training users to avoid the risks.

QUICK LOOK
autolog
The *autolog* package provides a program that automatically disconnects inactive users after a configurable delay. It also allows killing user processes that persist after a session ends, thereby preventing users from running daemons.